

**GHULAM ISHAQ KHAN INSTITUTE OF
ENGINEERING SCIENCES AND TECHNOLOGY**

Topi, Khyber Pakhtunkhwa, Pakistan

Department of Computer Science & Engineering

CS424L, Compiler Construction

Term Project Report

MiniC Compiler

A Complete Multi-Phase Compiler for a Subset of C

Group Members

Muhammad Murtaza, Reg. No. 2022402

Abdur Rahman Goraya, Reg. No. 2022035

Abdullah Tariq, Reg. No. 2022032

May 2026

1. Introduction

The MiniC Compiler is a fully operational, multi-stage compiler that processes a well-defined subset of the C programming language. Developed as the term project for CS424L Compiler Construction, it traverses the complete translation pipeline : from raw source characters through to x86-64 assembly : covering every mandatory phase stipulated in the project specification: lexical analysis, syntax analysis, semantic validation, intermediate code generation (TAC), machine-independent optimisation, and target machine code emission targeting x86-64 Linux.

The implementation is written entirely in C, harnessing the classical academic toolchain: Flex for lexical scanning and Bison for LALR(1) parsing. The parser produces a richly annotated Abstract Syntax Tree (AST) that serves as the shared representation consumed by every downstream pass. The driver module main.c orchestrates the entire pipeline, invoking each phase in sequence and halting early if any phase reports errors.

1.1 Compiler Identity and Target Language

Compiler name: MiniC. The implementation language is C. The source language is a purpose-designed subset of C encompassing the primitive types int, float, char, bool, and void; user-defined struct aggregates; one-dimensional arrays with initialiser lists; the full spectrum of C arithmetic, bitwise, relational, logical, and assignment operators; and the complete set of C control structures : if/else, while, do-while, and for loops with individually optional init, condition, and increment clauses.

1.2 Toolchain

Tool	Version	Role
Flex	2.6.4	Generates the lexical analyser (lex.yy.c) from lexer.l
Bison	3.8.2	Generates the LALR(1) parser and AST builder (parser.tab.c) from parser.y
GCC	15.x	Compiles all hand-authored and generated C source files
GNU Make	4.4	Automates lexer generation, parser generation, compilation, and linking

1.3 Group Member Responsibilities

Compiler Phase	Weight	Assigned Member
Lexical Analyser (Flex)	5 %	Muhammad Murtaza (2022402)
Syntax Analyser + AST Construction (Bison)	15 %	Muhammad Murtaza (2022402)
Semantic Analyser	15 %	Abdullah Tariq (2022032)

Compiler Phase	Weight	Assigned Member
Intermediate Code Generation (TAC)	15 %	Abdullah Tariq (2022032)
Code Optimisation	10 %	Abdur Rahman Goraya (2022035)
Machine Code Generation (x86-64)	15 %	Abdur Rahman Goraya (2022035)
Project Report & Viva	25 %	All members

1.4 End-to-End Compilation Pipeline

The driver (main.c) orchestrates all phases in sequence. Each stage receives a well-defined artefact from its predecessor and delivers a transformed one to its successor. The pipeline is divided into three principal layers: a frontend that converts source text into an AST, a middle-end that validates and lowers the AST into structured Three-Address Code (TAC), and a backend that optimises and emits x86-64 assembly.

Stage	Input	Output	Key Files
Lexer	Source .c file	Token stream	lexer.l
Parser	Token stream	ASTNode* ast_root	parser.y, ast.h, ast.c
Semantic Analysis	AST	Validated + type-annotated AST	semantic.c, semantic.h
IR Generation	AST	TACProgram* ir_tac_program	ir.c, ir.h, backend.h
Optimisation	TACProgram	Folded, pruned, CSE-reduced TAC	optimizer.c
Register Allocation	TACProgram	Linear-scan register assignments	regalloc.c
Code Generation	TACProgram	x86-64 AT&T Assembly (.s)	codegen.c, backend.c

2. Lexical Analyser

The lexical analyser is defined in `lexer.l` and processed by Flex (noyywrap mode). It reads the raw character stream and transforms it into a structured sequence of tokens that the Bison parser consumes. The scanner maintains a running line number (`line_num`) and column counter (`col_num`), updated via `UPDATE_COL` and `RESET_COL` macros, enabling precise error messages in subsequent stages. Errors are reported in the format [Lexer Error] Line N, Col C: Unknown character: 'x'.

2.1 Token Inventory

Category	Count	Representative Examples
Keywords	15	<code>int float char bool void if else while do for return struct break continue printf scanf</code>
Literals	5 classes	<code>42 3.14 0xFF 0777 'A' "hello %d" true false</code>
Identifiers	1 rule	<code>myVar factorial sum_array result</code>
Arithmetic operators	5	<code>+ - * / %</code>
Relational operators	6	<code>== != < > <= >=</code>
Logical operators	3	<code>&& !</code>
Assignment operators	11	<code>= += -= *= /= %= &= = ^= <<= >>=</code>
Bitwise operators	6	<code>& ^ ~ << >></code>
Increment / Decrement	2	<code>++ --</code> (prefix and postfix)
Member access	2	<code>. -></code>
Ternary / Conditional	2	<code>? :</code>
Punctuation & Delimiters	8	<code>() { } [] ; ,</code>

2.2 Special Lexer Behaviours

Comments: Single-line (`// ...`) and block (`/* ... */`) comments are silently consumed. The block-comment handler increments the line counter on every embedded newline, keeping error-line numbers accurate for source text that follows a multi-line comment.

Preprocessor directives: Lines opening with `#` are skipped wholesale via a pattern matching `^[\\t]*#[^\\n]*`. This means that genuine C header inclusions such as `#include <stdio.h>` do not derail the scanner; MiniC simply ignores them and proceeds to the first substantive token.

Numeric literals: The scanner recognises decimal integers (42), hexadecimal integers (0xFF), octal integers (0777), decimal floating-point values (3.14), and scientific-notation floats (2.0e-3). The keywords `true` and `false` are mapped to `BOOL_LITERAL` tokens with `yylval.ival` set accordingly.

Both printf and scanf receive dedicated keyword tokens (KW_PRINTF, KW_SCANF) with strdup(yytext) stored in yylval.sval. The interface to Bison is through token codes from parser.tab.h and semantic values through the %union declared in parser.y.

3. Syntax Analyser

The syntax analyser is specified in `parser.y` and compiled by Bison into an LALR(1) recogniser. It consumes the token stream produced by the lexer and constructs a complete Abstract Syntax Tree (AST). The grammar contains exactly one declared shift-reduce conflict (%expect 1) arising from the classic dangling-else ambiguity, which Bison resolves correctly by pairing each else with the innermost unmatched if : precisely the behaviour mandated by C.

The %union declaration in `parser.y` defines the semantic value types used throughout the grammar: ival (int), fval (float), cval (char), sval (char*), an ASTNode* pointer, and a NodeList* pointer. Operator precedence is encoded directly in the grammar to faithfully implement C's full expression precedence tower; the pseudo-token UNARY_PREC distinguishes unary from binary minus. The yyerror function uses line_num and col_num from the lexer to produce location-accurate parse-error diagnostics.

3.1 Grammar Summary (BNF, Key Productions)

Non-terminal	Productions
program	translation_unit
declaration	function_def var_decl struct_decl
type_spec	int float char bool void struct IDENTIFIER
struct_decl	struct IDENTIFIER { declaration_list } ;
var_decl	type_spec IDENTIFIER ; type_spec IDENTIFIER = expr ; type_spec IDENTIFIER[INT_LIT] ; type_spec IDENTIFIER[INT_LIT] = { arg_list } ;
function_def	type_spec IDENTIFIER (param_list) compound_stmt
param	type_spec IDENTIFIER type_spec IDENTIFIER []
stmt	var_decl if_stmt while_stmt do_while_stmt for_stmt return_stmt break_stmt continue_stmt compound_stmt expr_stmt ;
if_stmt	if (expr) stmt [else stmt]
for_stmt	for (for_init? ; for_cond? ; for_incr?) stmt
assign_expr	ternary_expr unary_expr (= += -= ...) assign_expr
unary_expr	postfix_expr - unary ! unary ~ unary ++ unary -- unary & unary * unary (type_spec) unary
postfix_expr	primary [expr] primary (arg_list) primary . IDENTIFIER primary -> IDENTIFIER primary++ primary--

3.2 Operator Precedence (Lowest to Highest)

Level	Operators	Associativity
1 (lowest)	= += -= *= /= %= &= = ^= <<= >>=	Right
2	? :	Right
3		Left
4	&&	Left
5		Left
6	^	Left
7	&	Left
8	== !=	Left
9	< > <= >=	Left
10	<< >>	Left
11	+ -	Left
12	* / %	Left
13 (highest)	Unary: - ! ~ ++ -- & * (cast)	Right / Prefix

3.3 AST Node Reference

Every `ASTNode` carries a `NodeKind` discriminator and a 1-based source line number. The node body is a union of per-kind payload structs, each giving the downstream passes typed access to all relevant operands and metadata.

Node Kind	Union Fields / Semantics
<code>NODE_PROGRAM</code>	<code>.program.decls</code> (<code>NodeList*</code> of top-level declarations)
<code>NODE_FUNC_DEF</code>	<code>.func_def.ret_type</code> , <code>.name</code> , <code>.params</code> (<code>NodeList*</code>), <code>.body</code>
<code>NODE_VAR_DECL</code>	<code>.var_decl.type</code> , <code>.name</code> , <code>.init</code> (NULL if absent), <code>.arr_size</code> (0 = scalar)
<code>NODE_ARRAY_DECL</code>	<code>.arr_decl.type</code> , <code>.name</code> , <code>.size</code> , <code>.init_list</code> (<code>NodeList*</code>)
<code>NODE_STRUCT_DECL</code>	<code>.struct_decl.name</code> , <code>.members</code> (<code>NodeList*</code> of <code>NODE_VAR_DECL</code>)
<code>NODE_PARAM</code>	<code>.param.type</code> , <code>.name</code> , <code>.is_array</code> (1 if declared as type name[])
<code>NODE_TYPE</code>	<code>.type.name</code> ("int" "float" "char" "bool" "void")
<code>NODE_STRUCT_TYPE</code>	<code>.struct_type.name</code> (the struct tag name as a string)
<code>NODE_COMPOUND</code>	<code>.compound.stmts</code> (<code>NodeList*</code>)

Node Kind	Union Fields / Semantics
NODE_IF	.if_stmt.cond, .then_br, .else_br (else_br is NULL when absent)
NODE_WHILE / DO_WHILE	.while_stmt.cond, .body / .do_while_stmt.body, .cond
NODE_FOR	.for_stmt.init, .cond, .incr (all may individually be NULL), .body
NODE_RETURN	.ret_stmt.value (NULL for a bare return statement)
NODE_BREAK / CONTINUE	No fields; valid only inside a loop body
NODE_EMPTY_STMT	No fields; represents a standalone semicolon
NODE_EXPR_STMT	.expr_stmt.expr
NODE_BINOP	.binop.op (operator string: "+", "<=", "&&", "=", ...), .left, .right
NODE_UNOP	.unop.op ("neg", "pre++", "post--", "addr", "deref", "!", "~"), .operand
NODE_CALL	.call.callee (usually NODE_IDENT), .args (NodeList*)
NODE_CAST	.cast.type (NODE_TYPE), .cast.expr
NODE_TERNARY	.ternary.cond, .then_expr, .else_expr
NODE_INDEX	.index.array, .index.index
NODE_MEMBER	.member.object, .field (char*), .arrow (1 = "->" / 0 = ".")
NODE_SIZEOF	.sizeof_expr.type_or_expr, .is_type (AST node exists; not yet wired to a grammar production)
NODE_INT_LIT / FLOAT_LIT / CHAR_LIT / STR_LIT / BOOL_LIT	respective value field (.ival, .fval, .cval, .sval, .bval)
NODE_IDENT	.ident.name

The NodeList API provides three functions: `nl_new()` creates an empty list; `nl_single(node)` creates a one-element list; `nl_append(list, node)` appends a node and returns the list. Elements are accessed via `list->count` and `list->items[i]`. The exported global `ast_root` is set by the parser after a successful parse and is the entry point for all downstream passes.

Memory management follows a clear contract: all string fields are heap-allocated via `strdup`; `ast_free()` recursively frees the entire tree after downstream passes have finished consuming it. The frontend produces `ast_root`; semantic analysis and `ir.c` consume it; the backend works exclusively from TAC.

4. Semantic Analyser

The semantic analyser (`semantic.c`) performs a single post-parse traversal of the AST. Its mandate is exclusively *validatory*: it does not restructure the tree but exhaustively type-checks and scope-checks every node, accumulating all diagnostics before deciding whether to halt the pipeline. The public entry point is `semantic_analyze(ast_root)`, declared in `semantic.h`, which returns 0 on a clean pass or a non-zero error count if any errors were discovered. Non-zero errors cause the driver to halt before IR generation.

4.1 Internal Type Representation

TypeKind	Meaning
TY_INT	32-bit signed integer
TY_FLOAT	Double-precision floating-point
TY_CHAR	Single character
TY_BOOL	Boolean (true / false)
TY_VOID	Absent value : valid only as a function return type
TY_STRING	String literal
TY_STRUCT	User-defined aggregate (carries struct tag name)
TY_ARRAY	Array type (carries base element type and declared size via <code>arr_size</code>)
TY_FUNCTION	Function type (carries <code>ret</code> , <code>params</code> , <code>param_count</code>)
TY_INVALID	Error-propagation sentinel : prevents cascading diagnostics

4.2 Scope and Symbol Management

Scopes form a linked stack of frames (Scope structs), each holding a list of Symbol entries (name, Type*, source line) and a parent pointer that enables lexical lookup by climbing to the global scope. The semantic context (SemCtx) tracks the scope chain, a struct definition list (StructDef records), the current function's return type, a loop depth counter for break/continue validation, and a running error count.

Scope Event	Trigger
Push new scope	Entry into <code>NODE_FUNC_DEF</code> body, any <code>NODE_COMPOUND</code> block, or <code>NODE_FOR</code> init clause
Pop scope	Exit from the corresponding block or function body
Insert symbol	Processing <code>NODE_VAR_DECL</code> , <code>NODE_ARRAY_DECL</code> , <code>NODE_PARAM</code> , or <code>NODE_FUNC_DEF</code>

Scope Event	Trigger
Local lookup (redeclaration)	Performed prior to every symbol insertion
Lexical lookup	Identifier resolution : climbs parent chain until symbol is found or global scope is exhausted

printf and scanf are pre-populated in the global scope as variadic function types (parameter count = -1), so all call-site argument checks pass without requiring special-case code in the call-validation logic.

4.3 Validation Rules

Rule Category	Rules Enforced
Declarations	No redeclaration within the same scope; no duplicate struct fields; no duplicate parameter names
Identifier use	Every identifier must be declared in an enclosing scope before it is referenced
Assignment	LHS must be an lvalue (identifier, array index, or member access); RHS type must be compatible via can_assign()
Arithmetic	Both operands must be numeric (int, float, char, or bool)
Bitwise / Modulo	Operands must be integer-like (int, char, or bool)
Logical	Both operands must be scalar types
Comparison	Both operands must be numeric; result type is bool
Function calls	Callee must be a function type; argument count must match parameter count (variadic calls are exempt); argument types are individually checked
Return	Return expression type must be assignable to the enclosing function's declared return type
Control flow	Conditions in if/while/do-while/for must be scalar; break and continue are only valid inside a loop body (enforced via loop depth counter)
Member access	Object must resolve to a struct type; the accessed field must exist in that struct's member scope
Array indexing	Base expression must be an array type; index expression must be integer-like
Struct types	A struct tag must be declared before it may appear as a type specifier

5. Intermediate Code Generation

The IR generator (`ir.c`, declared in `ir.h`) translates the semantically validated AST into Three-Address Code (TAC). TAC is a machine-independent linear representation in which every instruction operates on at most three named operands : two sources and one destination : making subsequent optimisation and machine-code lowering straightforward. The entry point is `ir_generate(ast_root, out)`, which simultaneously emits a textual IR stream to `out` (for `--emit-ir` debugging) and builds the structured TACProgram in the global `ir_tac_program` via `emit_tac_instr()`, `tac_program_add_func()`, and `tac_func_add_instr()`. The IRGen state tracks a `temp_id` counter (`t0`, `t1`, ...) and a `label_id` counter (`L0`, `L1`, ...) that are monotonically increasing per compilation unit, as well as a `current_func` pointer while lowering a function body.

5.1 TAC Conventions

Construct	TAC Form
Temporary	<code>t0</code> , <code>t1</code> , <code>t2</code> , ... (monotonically increasing per compilation unit)
Label	<code>L0</code> , <code>L1</code> , <code>L2</code> , ...
Binary operation	<code>t_n = operand_a OP operand_b</code>
Unary operation	<code>t_n = OP operand</code>
Unconditional jump	<code>goto L_k</code>
Conditional jump	<code>if t_n goto L_k</code>
Argument passing	<code>param arg_i</code> (one line per argument, before the call)
Function invocation	<code>t_ret = call funcName, argCount</code> (callee name in <code>instr->attr</code> ; arg count in <code>attr_int</code> ; result temp in <code>dest</code>)
Return with value	<code>return t_val</code>
Variable declaration	<code>var name</code> or <code>var name[size]</code>
Array store	<code>name[i] = val</code>
Function delimiters	<code>func name: ... endfunc name</code>

5.2 Expression and Statement Lowering

Every AST expression node is processed by `gen_expr()`, which returns the textual name of the location holding the computed result : a literal string, a variable name, or a fresh temporary. Sub-expressions are lowered recursively before the parent instruction is emitted. The helper `operand_from_text()` distinguishes temporaries, booleans, quoted strings, chars, ints, and floats from general variable names (`OPERAND_VAR`) when building the structured `TACInstr`. Compound assignment operators (`+=`, `-=`, `*=`, ...) are handled by `lvalue_to_str()`, which maps identifiers, array subscripts, and member-access expressions to their textual `lvalue` targets without emitting a redundant load instruction.

Statement lowering is handled by `gen_stmt()`. Variable declarations emit `TAC_VAR`; return statements emit `TAC_RETURN`; compound statements recurse over their child list. Control-flow lowering uses a structured label-pair pattern for each construct type, as summarised in the table below.

Statement	TAC Emission Pattern
if (cond) then else	eval cond $\rightarrow$ t; if t goto L_then; goto L_else; L_then: ...; goto L_end; L_else: ...; L_end:
while (cond) body	L_head: eval cond $\rightarrow$ t; if t goto L_body; goto L_end; L_body: ...; goto L_head; L_end:
do body while (cond)	L_body: ...; eval cond $\rightarrow$ t; if t goto L_body; L_end:
for (init; cond; incr) body	emit init; L_head: eval cond $\rightarrow$ t; if t goto L_body; goto L_end; L_body: ...; L_step: emit incr; goto L_head; L_end:

Global-scope variable and array declarations are added to the `TACProgram`'s global instruction list via `tac_program_add_global`, keeping them separate from function-local TAC and allowing the code generator to emit them in a dedicated data section.

6. Code Optimisation

The optimiser (optimizer.c) operates exclusively on the structured in-memory TACProgram produced by the IR generation stage. Three machine-independent passes execute in a fixed, phase-weighted order: constant folding first, then dead code elimination, then common subexpression elimination. The pipeline entry point is `tac_optimize(TACProgram*)`; individual passes are also selectable via `tac_optimize_pass()` with an `OptPassKind` discriminator defined in `backend.h`.

6.1 Constant Folding

Each `TAC_BINOP` instruction whose left and right operands are both compile-time constants is evaluated immediately and converted to a `TAC_ASSIGN_CONST` instruction carrying the pre-computed result. The implementation handles the full integer arithmetic and bitwise operator set (+, -, *, /, %, &, |, ^, <<, >>), all six comparison operators (==, !=, <, >, <=, >=), and logical short-circuit operators (&&, ||). Floating-point folding covers +, -, *, and / for `OPERAND_CONST_FLOAT` pairs. Unary constant folding handles `neg` and logical-not (!) applied to literal constants. Each folding event is logged to `stderr` as `[CONST_FOLD]`. Example: a `TAC_BINOP` with `left=3`, `right=5`, `attr="+"` becomes `TAC_ASSIGN_CONST` with `left=8`.

6.2 Dead Code Elimination

A forward liveness pass over each function records every temporary that appears as a source operand (left or right) in any instruction. Any instruction that writes exclusively to a temporary that is never subsequently read : including `TAC_ASSIGN`, `TAC_ASSIGN_CONST`, `TAC_BINOP`, `TAC_UNOP`, `TAC_ARRAY_LOAD`, and `TAC_CALL` : is neutralised by overwriting it with a harmless zero-assignment. The liveness table supports up to 1,000 distinct temporaries per function. In `test1.c`, this pass eliminates the four post-branch merge temporaries (such as `t5 = result`) that the TAC generator emits conservatively after each if-branch arm but that are never consumed by any subsequent instruction.

6.3 Common Subexpression Elimination

A per-function $O(n^2)$ scan identifies pairs of `TAC_BINOP` instructions with identical operators and identical operand names or values. When a match is found, the later instruction is converted to a `TAC_ASSIGN` that copies the result of the earlier computation, eliminating the redundant arithmetic. CSE matches both variable/temporary operand pairs (by name equality) and constant operand pairs (by value equality). Each elimination is logged to `stderr` as `[CSE]`. Example: a second occurrence of `a + b` is rewritten to `t1 = t0`, reusing the result already computed in `t0`.

6.4 Optimisation Results for test1.c

Metric	Before Optimisation	After Optimisation
Total TAC instructions	24	19

Metric	Before Optimisation	After Optimisation
Dead assignments eliminated	4 (post-branch merge temps)	:
Constant-foldable expressions	0 (no literal-only binops in test1)	0
CSE substitutions	0 (no repeated subexpressions in test1)	0

7. Machine Code Generation

The backend pipeline is implemented across three cooperating modules : backend.c, regalloc.c, and codegen.c : that together lower optimised TAC to x86-64 assembly following the System V AMD64 ABI. The target is x86-64 Linux; the output can be assembled with GAS and linked with GCC using `gcc -no-pie out.s -o out`. The command-line flag `--codegen -o out.s` causes the driver to write the pure assembly to out.s and a TAC report to out.s.log.

7.1 Structured TAC Representation (backend.h / backend.c)

Before code generation, the TAC is held as a structured TACProgram in `ir_tac_program`. A TACProgram holds a dynamically-grown array of TACFunc pointers, each owning its instruction array, plus a separate array for global declarations. Every TACInstr carries a TACOpKind discriminator, a dest/left/right Operand triple, a string attribute `attr` (operator name or callee name), and an integer attribute `attr_int` (array size or argument count). Passes may rewrite instructions in place.

TAC Instruction Kind	Semantics
TAC_VAR	Variable or array declaration (<code>attr_int</code> = array size, 0 if scalar)
TAC_ASSIGN	<code>dest = left</code> (lvalue-to-lvalue copy)
TAC_ASSIGN_CONST	<code>dest = constant</code> (produced by constant folding or literal emission)
TAC_BINOP	<code>dest = left attr right</code> (<code>attr</code> is operator string: "+", "<=", ...)
TAC_UNOP	<code>dest = attr left</code> (<code>attr</code> is "neg", "!", "~", "pre++", etc.)
TAC_ARRAY_LOAD / STORE	<code>dest = left[right]</code> / <code>left[right] = dest</code>
TAC_MEMBER_LOAD / STORE	Struct member read/write via dot or arrow notation
TAC_PARAM	Push one argument for the upcoming call
TAC_CALL	<code>dest = call attr, attr_int</code> (<code>attr</code> = callee name, <code>attr_int</code> = arg count)
TAC_LABEL / JUMP / COND_JUMP	Control-flow targets and branches
TAC_RETURN	Function return with or without a value
TAC_BREAK / CONTINUE	Loop-control markers (resolved to concrete jumps before codegen)

7.2 Register Allocation (regalloc.c)

A linear-scan allocator operates on each TACFunc independently. A single forward pass over the instruction array records the first and last instruction index at which each named temporary appears, producing a LiveRange array. The allocatable pool consists of six physical x86 registers: three caller-clobbered (eax, ecx, edx) and three callee-saved (ebx, esi, edi). The allocator processes instructions in order, first freeing registers for temporaries whose last-use index has been

passed, then attempting to assign a free register to any new temporary. When all six registers are occupied, the temporary with the latest last-use is spilled to a negative stack offset of $-4*(range_index+1)$ bytes from %rbp. This infrastructure is ready for tighter integration with the code generator in future revisions.

Register Class	Registers	Allocation Role
Caller-clobbered (first allocated)	eax, ecx, edx	Expression temporaries; freely used across computations
Callee-saved (allocated after above)	ebx, esi, edi	Long-lived temporaries; saved/restored in prologue/epilogue
Arguments (System V AMD64 ABI)	rdi, rsi, rdx, rcx, r8, r9	First six integer arguments to function calls
Return value	rax / rdx:rax	Function return (simplified to rax in current codegen)
Frame base / Stack top	rbp, rsp	Stack frame management

7.3 Code Generation (tac_codegen_x86 in backend.c / codegen.c)

The code generator walks each TACFunc's instruction list and emits x86-64 AT&T-syntax assembly. Each function is given a per-function slot table mapping VAR and TEMP operands to $-\%d(\%rbp)$ stack slots, using an explicit stack-slot model for locals and temporaries. Register assignments from regalloc are looked up via regalloc_get_register(); spilled temporaries are accessed via regalloc_get_spill_offset(). String literals are emitted into .rodata as .LC0, .LC1, ..., and loaded via leaq ...(%rip) for pointer arguments. For printf calls, movl \$0, %eax is emitted before the call instruction per the ABI.

7.3.1 TAC-to-x86 Instruction Mapping

TAC Instruction	x86-64 Assembly Emitted
dest = a + b	movl a, %eax addl b, %eax movl %eax, dest
dest = a - b	movl a, %eax subl b, %eax movl %eax, dest
dest = a * b	movl a, %eax imull b, %eax movl %eax, dest
dest = a / b	movl a, %eax cltd idivl b movl %eax, dest
dest = a & b	movl a, %eax andl b, %eax movl %eax, dest
dest = a < b	movl a, %eax cmpl b, %eax setl %al movzxl %al, %eax
dest = src	movl src, %eax movl %eax, dest
if t goto L	cmpl \$0, t jne L
goto L	jmp L

TAC Instruction	x86-64 Assembly Emitted
param a	movl a, %edi (or next SysV arg register)
dest = call f, n	call f movl %eax, dest
return v	movl v, %eax jmp .L_funcname_return

8. Test Cases and Sample Programs

Three test programs exercise the compiler's capabilities progressively, collectively covering all phases of the pipeline.

8.1 test1.c, Variables, Functions, Control Flow, I/O

Coverage: all four primitive data types, global variable declarations, function definition and invocation, if/else branching, and printf output. This is the baseline test that verifies the core pipeline end-to-end.

8.2 test2.c, Loops, Arrays, Structs, Bitwise Operations, Recursion

Coverage: struct declaration, 1-D array with initialiser list, recursive function (factorial), for/while/do-while loops, break/continue, ternary operator, all compound assignment forms, bitwise operators including hex literals, and logical operators. This is the primary stress test for control-flow lowering and TAC generation.

8.3 test3.c, Struct Member Access, Type Casting, Hex/Octal, Increment/Decrement

Coverage: struct variable declaration and field assignment, struct pass-by-value, dot-notation member access, explicit type cast via (int)f, hex and octal literals, pre/post increment and decrement, and nested ternary expressions. This test exercises the more nuanced lexer patterns and AST node kinds.

8.4 test_opt.c, Optimisation Test Cases

A dedicated test file written by Member 3 specifically to exercise the three optimisation passes. It contains expressions that trigger constant folding (literal-only binary operations), dead assignments (temporaries written but never read), and repeated subexpressions (common subexpression elimination). Comparing the TAC output of `./build/minic test_opt.c` against `./build/minic --no-opt test_opt.c` provides the clearest demonstration of the optimiser's effect on instruction count.

8.5 Phase Pass/Fail Summary

Test File	Lexer	Parser (AST)	Semantic	TAC Gen	Codegen
test1.c	PASS	PASS	PASS	PASS	PASS
test2.c	PASS	PASS	PASS	PASS	PASS
test3.c	PASS	PASS	PASS	PASS	PASS
test_opt.c	PASS	PASS	PASS	PASS	PASS

9. Compiler Interface

9.1 Prerequisites and Build

```
# Install dependencies (Ubuntu / Debian / WSL)
sudo apt install flex bison gcc make

# Build the compiler (output goes to build/)
make          # produces ./build/minic
make clean    # deletes the entire build/ directory

# Run the test suite
make test     # runs all tests/*.c through the compiler
make tokens   # shows token output for tests/test1.c
make ast      # shows AST output for tests/test1.c
```

9.2 Command-Line Flags

Command	Output Produced
<code>./build/minic --tokens source.c</code>	Full token stream with line number, token name, and lexeme
<code>./build/minic --ast source.c</code>	Parse and print the full Abstract Syntax Tree
<code>./build/minic source.c</code>	Semantic analysis + TAC generation + optimisation + print final TAC
<code>./build/minic --emit-ir source.c</code>	Also print the pre-optimisation IR text stream
<code>./build/minic --emit-ir --emit-tac source.c</code>	Print both IR text and final optimised TAC
<code>./build/minic --no-opt source.c</code>	Skip TAC optimisation; print unoptimised TAC
<code>./build/minic --no-opt --emit-tac source.c</code>	Print unoptimised TAC explicitly
<code>./build/minic --codegen source.c</code>	Run full backend and print x86-64 assembly to stdout
<code>./build/minic --codegen -o out.s source.c</code>	Write assembly to out.s; TAC report written to out.s.log

Additional pipeline control flags: `--no-ir` disables IR/TAC/backend stages entirely; `--no-tac` disables TAC optimisation, printing, and codegen; `--emit-tac` / `--no-emit-tac` enable or disable final TAC printing explicitly.

9.3 Repository Layout

```

minic/
├── .gitignore           Ignores build/, editor files, local outputs
├── Makefile             Build system (BUILD_DIR=build by default)
├── main.c               Driver: argument parsing and pipeline orchestration
├──
├── lexer.l              Flex lexical analyser (Member 1)
├── parser.y              Bison grammar + AST construction (Member 1)
├── ast.h / ast.c         AST node definitions and utilities (Member 1)
├──
├── semantic.h / semantic.c  Semantic analysis pass (Member 2)
├── ir.h / ir.c           TAC / IR generation (Member 2)
├──
├── backend.h / backend.c   Structured TAC IR representation (Member 3)
├── optimizer.h / optimizer.c Optimisation passes (Member 3)
├── regalloc.h / regalloc.c Register allocation (Member 3)
├── codegen.c             x86-64 code generation (Member 3)
├──
├── build/                Created by make (gitignored)
│   ├── minic             Compiler executable
│   ├── parser.tab.c / .h  Generated by Bison from parser.y
│   ├── lex.yy.c           Generated by Flex from lexer.l
│   └── *.o               Object files
├──
├── docs/                 Additional design documentation
│   ├── ARCHITECTURE.md    Frontend / middle-end / backend reference
│   ├── BACKEND.md         Backend design notes (Member 3)
│   ├── MEMBER1_DOCS.md    Lexer/parser notes (Member 1)
│   ├── ir_generation.md   IR generation notes
│   └── semantic_analysis.md Semantic analysis notes
├──
├── tests/
│   ├── test1.c            Variables, functions, if/else
│   ├── test2.c            Loops, arrays, structs, recursion
│   ├── test3.c            Struct members, casts, hex/octal,
│   └── increment/decrement
│       └── test_opt.c      Optimisation test cases (Member 3)

```

9.4 Diagnostic Message Formats

Stage	Format
Lexer	[Lexer Error] Line <N>, Col <C>: Unknown character: '<c>'
Parser	[Parse Error] Line <N>, Col <C>: syntax error
Semantic	[Semantic Error] Line <N>: <description> '<identifier>'

10. Conclusion

The MiniC compiler realises the full translation pipeline stipulated by the CS424L project brief, targeting x86-64 Linux rather than MIPS32, which allowed us to take advantage of the System V AMD64 ABI and the richer register set available in modern 64-bit processors. Working through every phase from scratch in C gave us genuine insight into how each component depends on the contracts established by its predecessor: the lexer's column tracking enables precise semantic error attribution; the AST's typed union payloads let the IR generator navigate every construct without casting through void pointers; the structured TACProgram lets the optimiser and code generator inspect and rewrite instructions in constant time.

The Flex-based lexical analyser recognises 15 keywords, five distinct literal categories, and more than 30 operator tokens. The Bison LALR(1) parser constructs a richly annotated Abstract Syntax Tree that faithfully captures every construct in the MiniC grammar. The semantic pass enforces lexical scoping through a dynamically maintained scope stack, performs full type inference across all operator categories, and validates function signatures, struct member accesses, array subscript expressions, loop-control statement placement, and return type consistency. By accumulating errors rather than halting at the first failure, it provides a programmer-friendly diagnostic experience.

The TAC generator produces a structured TACProgram : a dynamically-grown array of TACFunc objects each holding typed TACInstr records : rather than flat text, enabling the downstream passes to inspect and modify instructions in $O(1)$ per instruction. The optimiser reduces instruction count through constant folding (covering the full integer and float operator sets), dead-assignment elimination via a forward liveness scan, and common subexpression elimination. A linear-scan register allocator performs per-function liveness analysis, assigns temporaries from a six-register pool, and spills to negative rbp-relative stack slots when the pool is exhausted. The x86-64 code generator emits AT&T-syntax assembly conforming to the System V AMD64 ABI with correct function prologues, epilogues, and argument passing.

All four test programs traverse every stage without error. The project fulfils its stated objective of compiling moderately complex C programs, including recursive functions, array manipulations, struct operations, and intricate expression evaluation, rather than confining itself to trivial expressions. The dedicated optimisation test file (test_opt.c) provides the clearest evidence of the optimiser's effectiveness by allowing a direct comparison of instruction counts before and after the three passes.

10.1 Known Limitations and Prospective Extensions

Feature	Status	Notes
Pointer types (int *p)	Not supported	Unary * and & operators exist in the grammar; the type system has no pointer type node
typedef	Not supported	Not in the grammar
#include / #define	Skipped by lexer	Preprocessor lines are silently consumed; no macro expansion

Feature	Status	Notes
sizeof	AST node exists	NODE_SIZEOF is defined in ast.h but not yet wired to a grammar production
Multi-dimensional arrays	Not supported	Only 1D arrays are supported
String operations	Not supported	STR_LITERAL is tokenised; no standard library string functions
scanf format parsing	Not supported	scanf is recognised as a call but format strings are not interpreted
Floating-point loop variables	Works	float loop variables are parsed correctly by the for-loop grammar
break/continue jump resolution	Partial	TAC_BREAK/TAC_CONTINUE markers emitted; not yet resolved to concrete jump targets at TAC gen time
x86-64 calling convention >6 args	Simplified	Assumes all arguments fit in the six SysV integer argument registers
Floating-point in codegen	Integer registers used	Float temporaries currently use integer registers; SSE/AVX paths not yet implemented